DBMS Laboratory (24U0446) - Practical Programs

Only C2, C3, C4, D1, D3, D4, E1, E2 and E4. Short, tested programs without comments. SQL and PL/SQL-style programs run in MySQL, and the MongoDB scripts run in mongosh.

Group	Practicals	Tool
C (Unit III)	C2 domain constraints, C3 referential integrity, C4 3NF	MySQL
D (Unit IV)	D1 ACID and transaction states, D3 view serializability, D4 timestamp ordering	MySQL
E (Unit V)	E1 document design, E2 CRUD, E4 aggregation	mongosh

To run: SQL files with `mysql -u root -p < file.sql`, MongoDB files by pasting into `mongosh`.

GROUP C - SQL (MySQL)

C2. Implementation of Domain Constraints

NOT NULL, CHECK, UNIQUE and DATE checks. The first two inserts succeed; the last five are meant to fail, one per constraint.

```
CREATE DATABASE empdb;
USE empdb;

CREATE TABLE employee (
  emp_id INT PRIMARY KEY,
  emp_name VARCHAR(30) NOT NULL,
  salary DECIMAL(10,2) NOT NULL CHECK (salary > 0),
  email VARCHAR(50) UNIQUE CHECK (email LIKE '%_@_%._%'),
  join_date DATE NOT NULL CHECK (join_date >= '2000-01-01')
);

INSERT INTO employee VALUES (1,'Amit',50000,'amit@mail.com','2023-06-01');
INSERT INTO employee VALUES (2,'Sara',60000,'sara@mail.com','2024-01-15');

INSERT INTO employee VALUES (3,'Ravi',-100,'ravi@mail.com','2024-02-01');
INSERT INTO employee VALUES (4,'Neha',45000,'amit@mail.com','2024-02-01');
INSERT INTO employee VALUES (1,'Om',45000,'om@mail.com','2024-02-01');
INSERT INTO employee VALUES (5,'Kiran',45000,'kiran.com','2024-02-01');
INSERT INTO employee VALUES (6,'Jay',45000,'jay@mail.com','1990-01-01');

SELECT * FROM employee;
```

C3. Implementation of Referential Integrity Constraints

ON UPDATE CASCADE changes sid 2 to 5 in enrollment. ON DELETE CASCADE removes the student's enrollments. ON DELETE RESTRICT blocks deleting a course that is in use. The insert of student 9 fails (no parent row).

```
CREATE DATABASE regdb;
USE regdb;

CREATE TABLE student (
  sid INT PRIMARY KEY,
  sname VARCHAR(30) NOT NULL
);

CREATE TABLE course (
  cid INT PRIMARY KEY,
  cname VARCHAR(30) NOT NULL
);

CREATE TABLE enrollment (
  sid INT,
  cid INT,
  PRIMARY KEY (sid, cid),
  FOREIGN KEY (sid) REFERENCES student(sid)
    ON DELETE CASCADE ON UPDATE CASCADE,
  FOREIGN KEY (cid) REFERENCES course(cid)
    ON DELETE RESTRICT ON UPDATE CASCADE
);

INSERT INTO student VALUES (1,'Amit'),(2,'Sara');
```

```
INSERT INTO course VALUES (101,'DBMS'),(102,'OS');
INSERT INTO enrollment VALUES (1,101),(1,102),(2,101);

INSERT INTO enrollment VALUES (9,101);

UPDATE student SET sid = 5 WHERE sid = 2;
SELECT * FROM enrollment;

DELETE FROM student WHERE sid = 1;
SELECT * FROM enrollment;

DELETE FROM course WHERE cid = 101;
```

C4. Normalization to Third Normal Form (3NF)

Transitive dependency: emp_id -> dept_id -> dept_name, dept_loc. Split into emp and dept. The final join proves the decomposition is lossless.

```
CREATE DATABASE normdb;
USE normdb;

CREATE TABLE emp_all (
  emp_id INT PRIMARY KEY,
  emp_name VARCHAR(30),
  dept_id INT,
  dept_name VARCHAR(30),
  dept_loc VARCHAR(30)
);

INSERT INTO emp_all VALUES
(1,'Amit',10,'Sales','Pune'),
(2,'Sara',10,'Sales','Pune'),
(3,'Ravi',20,'HR','Nashik');

CREATE TABLE dept (
  dept_id INT PRIMARY KEY,
  dept_name VARCHAR(30),
  dept_loc VARCHAR(30)
);

INSERT INTO dept
SELECT DISTINCT dept_id, dept_name, dept_loc FROM emp_all;

CREATE TABLE emp (
  emp_id INT PRIMARY KEY,
  emp_name VARCHAR(30),
  dept_id INT,
  FOREIGN KEY (dept_id) REFERENCES dept(dept_id)
);

INSERT INTO emp SELECT emp_id, emp_name, dept_id FROM emp_all;

SELECT e.emp_id, e.emp_name, d.dept_name, d.dept_loc
FROM emp e JOIN dept d ON e.dept_id = d.dept_id;
```

GROUP D - Transactions (MySQL)

D1. Demonstration of Transaction States and ACID Properties

States: Active, Partially Committed, Committed, Failed, Aborted. COMMIT makes the transfer permanent (Durability). ROLLBACK undoes it (Atomicity). The CHECK on balance keeps data valid (Consistency). The procedure rolls back automatically on any error.

```
CREATE DATABASE bankdb;
USE bankdb;

CREATE TABLE account (
  acc_no INT PRIMARY KEY,
  holder VARCHAR(30),
  balance DECIMAL(10,2) CHECK (balance >= 0)
) ENGINE=InnoDB;

INSERT INTO account VALUES (101,'Amit',5000),(102,'Sara',3000);

START TRANSACTION;
UPDATE account SET balance = balance - 1000 WHERE acc_no = 101;
UPDATE account SET balance = balance + 1000 WHERE acc_no = 102;
SELECT * FROM account;
COMMIT;

START TRANSACTION;
UPDATE account SET balance = balance - 500 WHERE acc_no = 101;
SELECT * FROM account;
ROLLBACK;
SELECT * FROM account;

DELIMITER //
CREATE PROCEDURE transfer(IN src INT, IN dst INT, IN amt DECIMAL(10,2))
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK;
    SELECT 'FAILED - rolled back' AS status;
  END;
  START TRANSACTION;
  UPDATE account SET balance = balance - amt WHERE acc_no = src;
  UPDATE account SET balance = balance + amt WHERE acc_no = dst;
  COMMIT;
  SELECT 'SUCCESS - committed' AS status;
END //
DELIMITER ;

CALL transfer(101, 102, 500);
CALL transfer(101, 102, 99999);
SELECT * FROM account;
```

D3. View Serializability Analysis

Schedule 0 is the given one; orders 1 to 6 are all serial orders of T1, T2, T3. The query lists serial orders with the same reads-from and same final writes. The schedule has blind writes, so it is view serializable (order 1: T1,T2,T3).

```
CREATE DATABASE viewdb;
USE viewdb;

CREATE TABLE sched (
  oid INT,
  seq INT,
  txn VARCHAR(5),
  op CHAR(1),
  item VARCHAR(20),
  PRIMARY KEY (oid, seq)
);

INSERT INTO sched VALUES
(0,1,'T1','R','course'),(0,2,'T2','W','course'),
(0,3,'T1','W','course'),(0,4,'T3','W','course');

CREATE TABLE perm (oid INT, pos INT, txn VARCHAR(5));
INSERT INTO perm VALUES
(1,1,'T1'),(1,2,'T2'),(1,3,'T3'),
```

```
(2,1,'T1'),(2,2,'T3'),(2,3,'T2'),
(3,1,'T2'),(3,2,'T1'),(3,3,'T3'),
(4,1,'T2'),(4,2,'T3'),(4,3,'T1'),
(5,1,'T3'),(5,2,'T1'),(5,3,'T2'),
(6,1,'T3'),(6,2,'T2'),(6,3,'T1');

INSERT INTO sched
SELECT p.oid, p.pos * 100 + s.seq, s.txn, s.op, s.item
FROM sched s JOIN perm p ON s.txn = p.txn
WHERE s.oid = 0;

CREATE VIEW reads_from AS
SELECT r.oid, r.txn, r.item,
  COALESCE((SELECT w.txn FROM sched w
            WHERE w.oid = r.oid AND w.item = r.item AND w.op = 'W' AND w.seq < r.seq
            ORDER BY w.seq DESC LIMIT 1), 'init') AS src
FROM sched r WHERE r.op = 'R';

CREATE VIEW final_write AS
SELECT w.oid, w.item, w.txn
FROM sched w
WHERE w.op = 'W'
  AND w.seq = (SELECT MAX(x.seq) FROM sched x
               WHERE x.oid = w.oid AND x.item = w.item AND x.op = 'W');

SELECT o.oid AS serial_order
FROM (SELECT DISTINCT oid FROM sched WHERE oid > 0) o
WHERE NOT EXISTS (SELECT 1 FROM reads_from a
                  WHERE a.oid = 0 AND NOT EXISTS
                    (SELECT 1 FROM reads_from b WHERE b.oid = o.oid
                     AND b.txn = a.txn AND b.item = a.item AND b.src = a.src))
  AND NOT EXISTS (SELECT 1 FROM final_write a
                  WHERE a.oid = 0 AND NOT EXISTS
                    (SELECT 1 FROM final_write b WHERE b.oid = o.oid
                     AND b.item = a.item AND b.txn = a.txn));

SELECT oid, GROUP_CONCAT(txn ORDER BY pos) AS serial_order FROM perm GROUP BY oid;
```

D4. Timestamp-Based Concurrency Control

Read: rollback if TS(T) < W-TS. Write: rollback if TS(T) < R-TS or TS(T) < W-TS. Otherwise execute and update the timestamps. Call do_op for each operation of the schedule.

```
CREATE DATABASE tsdb;
USE tsdb;

CREATE TABLE txn (name VARCHAR(5) PRIMARY KEY, ts INT);
CREATE TABLE item (name VARCHAR(10) PRIMARY KEY, rts INT, wts INT, val VARCHAR(20));

INSERT INTO txn VALUES ('T1',1),('T2',2),('T3',3);
INSERT INTO item VALUES ('ans',0,0,'none');

DELIMITER //
CREATE PROCEDURE do_op(IN t VARCHAR(5), IN o CHAR(1), IN x VARCHAR(10), IN v VARCHAR(20))
BEGIN
  DECLARE tts INT;
  DECLARE r INT;
  DECLARE w INT;
  SELECT ts INTO tts FROM txn WHERE name = t;
  SELECT rts, wts INTO r, w FROM item WHERE name = x;
  IF o = 'R' THEN
    IF tts < w THEN
      SELECT CONCAT(t, ' READ ', x, ' -> ROLLBACK') AS result;
    ELSE
      UPDATE item SET rts = GREATEST(rts, tts) WHERE name = x;
      SELECT CONCAT(t, ' READ ', x, ' -> OK') AS result;
    END IF;
  ELSE
    IF tts < r OR tts < w THEN
      SELECT CONCAT(t, ' WRITE ', x, ' -> ROLLBACK') AS result;
    ELSE
      UPDATE item SET wts = tts, val = v WHERE name = x;
      SELECT CONCAT(t, ' WRITE ', x, ' -> OK') AS result;
    END IF;
  END IF;
END //
DELIMITER ;
```

```
CALL do_op('T2','R','ans',NULL);
CALL do_op('T3','W','ans','B');
CALL do_op('T1','W','ans','A');
CALL do_op('T2','W','ans','C');
CALL do_op('T1','R','ans',NULL);

SELECT * FROM item;
```

GROUP E - MongoDB (run in mongosh)

E1. Design of a Document Database for an Online Book Store

Embedded author document, genres array and reviews array. Documents may have different fields (pages exists only on the second book).

```
use bookstore

db.books.insertMany([
  {
    title: "Dune",
    author: { name: "Frank Herbert", country: "USA" },
    price: 450,
    genres: ["scifi", "classic"],
    reviews: [
      { user: "amit", rating: 5, comment: "Great" },
      { user: "sara", rating: 4 }
    ]
  },
  {
    title: "Wings of Fire",
    author: { name: "APJ Abdul Kalam" },
    price: 300,
    pages: 180,
    reviews: []
  }
])

db.books.find()
db.books.find({ "author.name": "Frank Herbert" })
db.books.find({ genres: "scifi" })
db.books.find({ "reviews.rating": 5 }, { title: 1, _id: 0 })
db.books.updateOne({ title: "Dune" }, { $push: { reviews: { user: "ravi", rating: 5 } } })
db.books.find({ title: "Dune" }, { reviews: 1, _id: 0 })
```

E2. MongoDB CRUD Operations

Create: insertOne, insertMany. Read: find, findOne. Update: updateOne, updateMany. Delete: deleteOne, deleteMany.

```
use studentdb

db.students.insertOne({ roll: 1, name: "Amit", dept: "Computer", marks: 85 })
db.students.insertMany([
  { roll: 2, name: "Sara", dept: "Computer", marks: 92 },
  { roll: 3, name: "Ravi", dept: "Mechanical", marks: 70 }
])

db.students.find()
db.students.find({ dept: "Computer" })
db.students.find({ marks: { $gt: 80 } })
db.students.findOne({ roll: 3 })

db.students.updateOne({ roll: 3 }, { $set: { marks: 75 } })
db.students.updateMany({ dept: "Computer" }, { $inc: { marks: 5 } })

db.students.deleteOne({ roll: 3 })
db.students.deleteMany({ marks: { $lt: 50 } })

db.students.find()
```

E4. MongoDB Aggregation Framework for Sales Analysis

Monthly sales, top 3 selling products and category-wise revenue using $group, $sort, $limit and $multiply.

```
use retail

db.sales.insertMany([
  { product: "Pen",    category: "Stationery",  qty: 100, price: 10,    date: new Date("2026-01-10") },
  { product: "Book",   category: "Stationery",  qty: 40,  price: 150,   date: new Date("2026-01-20") },
  { product: "Phone",  category: "Electronics", qty: 5,   price: 15000, date: new Date("2026-02-05") },
  { product: "Pen",    category: "Stationery",  qty: 200, price: 10,    date: new Date("2026-02-15") },
  { product: "Laptop", category: "Electronics", qty: 2,   price: 50000, date: new Date("2026-03-01") }
])

db.sales.aggregate([
```

```
  { $group: { _id: { $month: "$date" }, total: { $sum: { $multiply: ["$qty", "$price"] } } } },
  { $sort: { _id: 1 } }
])

db.sales.aggregate([
  { $group: { _id: "$product", sold: { $sum: "$qty" } } },
  { $sort: { sold: -1 } },
  { $limit: 3 }
])

db.sales.aggregate([
  { $group: { _id: "$category", revenue: { $sum: { $multiply: ["$qty", "$price"] } } } },
  { $sort: { revenue: -1 } }
])
```